

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ №5

ТЕМА: WYSIWYG редактор на базе ImGUI

ЦЕЛЬ

Разработать визуальный редактор сцены для игрового движка, интегрировать библиотеку Dear ImGui или другую совместимую (FLTK, Nuklear, GTK+ и т.п.), создать инструменты инспектирования и редактирования компонентов ECS в реальном времени (live preview), реализовать манипуляторы трансформаций (гизмо) и панель отображения статистики.

ЗАДАЧИ

1. Интегрировать библиотеку Dear ImGui в существующий движок.
2. Создать базовое меню и панели инструментов (тестовая отрисовка ImGui).
3. Реализовать панель инспектора для просмотра и редактирования компонентов выбранной сущности.
4. Разработать панель иерархии сцены (список всех сущностей).
5. Реализовать редактор сцены с live preview (изменения в инспекторе сразу видны на экране).
6. Внедрить манипуляторы трансформаций (гизмо) для перемещения, вращения и масштабирования объектов в 3D-пространстве.
7. Создать панель статистики (FPS, количество объектов, время отрисовки).
8. Обеспечить переключение между режимом редактирования и игровым режимом.

ГЛОССАРИЙ

Термин	Определение
ImGUI (Dear ImGui)	Библиотека для создания графических интерфейсов в режиме немедленного рендеринга (immediate mode).
WYSIWYG	What You See Is What You Get — режим редактирования, при котором изменения отображаются сразу в окне предпросмотра.
Инспектор	Панель, отображающая свойства выбранного объекта для редактирования.
Иерархия сцены	Список всех объектов сцены с поддержкой родительско-дочерних связей.
Гизмо (Gizmo)	Визуальный манипулятор (стрелки, окружности, квадраты) для трансформации объектов в 3D.
Live Preview	Возможность видеть результаты редактирования параметров в реальном времени без перезапуска.

Immediate Mode GUI	Режим GUI, при котором интерфейс полностью перерисовывается каждый кадр кодом приложения.
--------------------	---

НЕОБХОДИМЫЕ ИНСТРУМЕНТЫ

Шаг	Инструмент	Назначение	Обоснование
1-8	Dear ImGui	Основная библиотека GUI	Стандарт для инструментов разработки игр, легковесная, immediate mode
1	ImGui backends	Интеграция с GLFW/SDL и OpenGL/Vulkan	Официальные бэкенды обеспечивают корректную обработку ввода и рендеринг
2-5	ImGui::	Создание окон, кнопок, полей ввода	Стандартные виджеты ImGui
6	ImGuizmo (отдельная библиотека)	3D-манипуляторы	Решение от автора ImGui для трансформаций в 3D-пространстве
6	GLM	Математика для преобразований гизмо	Используется для работы с матрицами
4	ECS Registry	Получение списка сущностей	Доступ к данным сцены через существующую ECS-архитектуру
7	std::chrono	Замер времени для статистики	Точный подсчёт FPS и времени кадра

КРАТКАЯ ТЕОРЕТИЧЕСКАЯ СПРАВКА

Dear ImGui — библиотека для создания графических интерфейсов в режиме немедленного рендеринга (immediate mode) . В отличие от традиционных библиотек (Qt, wxWidgets), где интерфейс строится один раз и затем обновляется по событиям, в ImGui код интерфейса выполняется каждый кадр. Это даёт простоту интеграции и гибкость, особенно ценную для инструментов разработки игр.

ImGuizmo - <https://github.com/CedricGuillemet/ImGuizmo?ysclid=mmmkjqnb598109552>

Архитектура ImGui в игровом движке:

1. Инициализация контекста ImGui.
2. Инициализация бэкендов для платформы (GLFW/SDL) и рендерера (OpenGL/Vulkan).
3. В игровом цикле: начало кадра ImGui → создание интерфейса → рендеринг.
4. Завершение работы и очистка.

WYSIWYG-редактирование означает, что изменения, вносимые в инспекторе (например, изменение позиции объекта), немедленно отражаются в окне предпросмотра без необходимости перезапуска сцены. Это достигается за счёт прямой модификации компонентов ECS и перерисовки сцены в каждом кадре.

Гизмо (Gizmo) — интерактивные 3D-элементы управления, позволяющие перемещать, вращать и масштабировать объекты мышью непосредственно в окне предпросмотра. Библиотека ImGuizmo предоставляет готовую реализацию таких манипуляторов, работающую поверх ImGui.

ИСХОДНЫЕ ДАННЫЕ (с учётом предыдущих работ)

Имеется полностью функционирующий движок, включающий:

- Каркас приложения (Application) с игровым циклом и корректным deltaTime.
- Абстрактный RenderAdapter и его реализацию для выбранного графического API.
- ECS-архитектуру: World, сущности, компоненты Transform, Tag, MeshRenderer, Rigidbody, BoxCollider.
- ResourceManager с поддержкой загрузки моделей, текстур, шейдеров.
- PhysicsSystem с гравитацией и AABB-коллизиями, debug-отрисовкой.
- InputManager и систему событий.
- Возможность отрисовки нескольких 3D-объектов с текстурами.

Все перечисленные компоненты будут использованы и расширены в данной работе.

ЗАДАНИЕ И ПОРЯДОК ВЫПОЛНЕНИЯ

Шаг 1. Интеграция Dear ImGui в проект

Что сделать:

- Скачать исходные файлы Dear ImGui с GitHub (ветка docking рекомендуется для поддержки стыкуемых окон). <https://github.com/ocornut/imgui>
<https://github.com/ZephyrWolf/imgui-docking?ysclid=mmmkggot9730646881>
- Добавить в проект необходимые файлы: `imgui.cpp`, `imgui.h`, `imgui_draw.cpp`, `imgui_tables.cpp`, `imgui_widgets.cpp`, а также файлы бэкендов для вашей платформы и рендера (например, `imgui_impl_glfw.cpp` и `imgui_impl_opengl3.cpp` для GLFW+OpenGL).
- Настроить include-пути и линковку.

Инструменты: Браузер для скачивания, система сборки (CMake/VS).

Критический момент: убедитесь, что версии бэкендов соответствуют используемым версиям GLFW/SDL и графического API.

Результат: Проект компилируется с включёнными файлами ImGui.

Шаг 2. Инициализация и отрисовка тестового окна ImGui

Что сделать:

- В методе Application::Init() добавить:

```
ImGui_CHECKVERSION();
ImGui::CreateContext();
ImGuiIO& io = ImGui::GetIO(); (void)io;
io.ConfigFlags |= ImGuiConfigFlags_NavEnableKeyboard;
io.ConfigFlags |= ImGuiConfigFlags_DockingEnable; // если нужен docking
```

- Инициализировать бэкенды платформы и рендерера:

```
ImGui_ImplGlfw_InitForOpenGL(window, true);
ImGui_ImplOpenGL3_Init("#version 330");
```

- В игровом цикле, после очистки буферов, но до основного рендеринга сцены:

```
    ImGui_ImplOpenGL3_NewFrame();
    ImGui_ImplGlfw_NewFrame();
    ImGui::NewFrame();
```

- Создать тестовое окно:

```
ImGui::ShowDemoWindow(); // встроенная демонстрация
```

- После рендеринга сцены:

```
ImGui::Render();
ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());
```

- В Shutdown():

```
ImGui_ImplOpenGL3_Shutdown(); ImGui_ImplGlfw_Shutdown(); ImGui::DestroyContext();
```

Инструменты: Функции ImGui, бэкенды.

Результат: на экране поверх 3D-сцены отображается демо-окно ImGui с множеством виджетов.

Шаг 3. Создание главного меню и базовых окон

Что сделать:

Заменить ShowDemoWindow() на собственные окна.

Создать главное меню (Main Menu Bar) с пунктами File, View, Help.

Создать пустые окна: "Scene Hierarchy", "Inspector", "Statistics", "Viewport" (окно предпросмотра).

Настроить docking, чтобы окна можно было перемещать и стыковать.

Инструменты: ImGui::BeginMainMenuBar(), ImGui::BeginMenu(), ImGui::MenuItem(), ImGui::EndMenu(), ImGui::End().

Результат: Интерфейс редактора с пустыми панелями в стиле профессиональных IDE.

Шаг 4. Панель иерархии сцены (Scene Hierarchy)

Что сделать:

- Получить список всех сущностей из ECS World.
- Для каждой сущности отобразить её имя (из компонента Tag) в виде кликабельного элемента списка (ImGui::Selectable).
- Подсветить выбранную сущность.
- Поддержать вложенность (если есть компонент Hierarchy).

Инструменты: ECS-реестр, ImGui::Selectable, ImGui::SetNextItemOpen() для сворачивания.

Результат: В левой панели отображается список всех объектов сцены, можно выбирать объекты кликом.

Шаг 5. Панель инспектора (Inspector)

Что сделать:

- При выборе сущности в иерархии отобразить её компоненты.
- Для каждого компонента создать группу (TreeNode) с заголовком типа компонента.
- Реализовать редактирование полей:

Transform: позиция (vec3), поворот (vec3 углы или кватернион), масштаб (vec3) с помощью ImGui::DragFloat3.

Tag: редактируемое текстовое поле (ImGui::InputText).

MeshRenderer: выбор меша из загруженных ресурсов (комбобокс), цвет, выбор текстуры.

Rigidbody: флаги useGravity, isKinematic, скорость (только просмотр).

BoxCollider: размер, смещение.

Инструменты: ImGui::DragFloat3, ImGui::InputText, ImGui::Combo, ImGui::Checkbox, ImGui::ColorEdit3 (если нужно).

Критический момент: Изменения должны немедленно применяться к компонентам (это уже live preview, так как сцена перерисовывается каждый кадр).

Результат: при выборе объекта в правой панели отображаются его свойства, которые можно редактировать.

Шаг 6. Редактор сцены с live preview

Что сделать:

- Окно "Viewport" должно занимать основную область редактора.
- В этом окне отображается текущая 3D-сцена (через существующий рендеринг).

- Убедиться, что изменения в инспекторе (например, перемещение объекта) сразу видны в viewport — это уже работает, так как рендеринг использует актуальные данные компонентов.

Дополнительно: добавить возможность переключения между режимом "Edit" и "Play". В режиме "Play" физика и ввод работают как в игре, в режиме "Edit" — физика отключена, можно редактировать объекты.

Инструменты: Флаг состояния в Application, условное обновление систем.

Результат: Полноценный live-редактор сцены.

Шаг 7. Интеграция гизмо (манипуляторов трансформаций)

Что сделать:

- Подключить библиотеку ImGuizmo (один .h и один .cpp файл).
- В окне viewport, после установки камеры, но до отрисовки гизмо:
 - Получить матрицы вида и проекции от текущей камеры.
 - Для выбранной сущности получить её матрицу трансформации (из компонента Transform).
 - Вызвать ImGuizmo::Manipulate() с соответствующими параметрами.
 - После манипуляции обновить компонент Transform (позицию, поворот, масштаб) из результирующей матрицы.
- Добавить кнопки на панели инструментов для переключения режима гизмо (перемещение/вращение/масштабирование) и системы координат (локальная/глобальная).

Инструменты: ImGuizmo.h, GLM для преобразований матриц.

Результат: при выборе объекта вокруг него появляются стрелки/окружности, с помощью которых можно перемещать, вращать и масштабировать объект мышью прямо в 3D-виде.

Шаг 8. Панель статистики

Что сделать:

- Создать окно "Statistics".
- Отображать:
 - FPS (средний за последнюю секунду) — рассчитать через deltaTime.
 - Время кадра в миллисекундах.
 - Количество отрисованных объектов (сущностей с MeshRenderer).
 - Количество сущностей всего.
 - Количество активных коллизий (если есть).

Использование памяти ресурсами (можно вывести из ResourceManager).

Инструменты: std::chrono, счётчики в системах, ImGui::Text.

Результат: В углу экрана отображается полезная отладочная информация.

Шаг 9. Переключение между режимами Edit/Play

Что сделать:

- Добавить кнопки "Play" и "Stop" на панели инструментов.
- При нажатии "Play":
 - Сохранить текущее состояние сцены (опционально).
 - Включить обновление PhysicsSystem и обработку ввода.
 - Отключить редактирование компонентов (или оставить, но с осторожностью).
- При нажатии "Stop":
 - Восстановить сохранённое состояние (или просто перезагрузить сцену из исходных данных).

Инструменты: Состояние в Application, сериализация (можно временную).

Результат: Возможность тестировать игровую логику прямо в редакторе.

ЗАДАНИЕ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

1. **Сохранение/загрузка сцены из редактора:** добавить пункты меню File → Save/Load, которые сериализуют текущую сцену в JSON (используя наработки из ПЗ №3).
2. **Undo/Redo:** реализовать простую систему отмены действий (командный паттерн) для операций в инспекторе.
3. **Material Editor:** создать отдельную панель для редактирования материалов (цвета, текстуры, шейдеры) с предпросмотром на сфере/кубе.
4. **Asset Browser:** Панель для просмотра загруженных ресурсов (моделей, текстур) с возможностью перетаскивания их в сцену для создания новых объектов.
5. **Кастомизация темы ImGui:** создать собственный стиль (цвета, шрифты) через ImGui::StyleColorsDark() с модификациями.

РЕКОМЕНДАЦИИ ПО ВЫПОЛНЕНИЮ РАБОТЫ

- **Порядок инициализации:** ImGui должен инициализироваться после создания окна и графического контекста, но до основного игрового цикла.
- **Рендеринг:** Вызов ImGui::Render() должен происходить после отрисовки всей 3D-сцены, чтобы GUI был поверх изображения.

- **Обработка ввода:** Бэкенды ImGui автоматически обрабатывают ввод мыши/клавиатуры, но нужно убедиться, что события не "поедаются" движком. Обычно ImGui перехватывает ввод, если курсор над его окнами (`io.WantCaptureMouse`, `io.WantCaptureKeyboard`). Используйте эти флаги, чтобы отключать игровой ввод при работе с редактором.
- **Гизмо:** ImGuiizo требует корректных матриц вида и проекции. Убедитесь, что камера правильно передаёт эти матрицы. После манипуляции не забудьте обновить компонент Transform.
- **Производительность:** ImGui очень лёгкий, но панели со сложными списками (например, иерархия на 1000 объектов) могут тормозить. Используйте виртуальные списки (`ImGuiListClipper`) при необходимости.
- **Командная работа:** распределите задачи: один участник занимается интеграцией ImGui и базовыми окнами, второй — инспектором и связью с ECS, третий — гизмо и viewport, четвёртый — статистикой и режимами Edit/Play.

ПОСЛЕДОВАТЕЛЬНОСТЬ ВЫПОЛНЕНИЯ

1. Интеграция ImGui, инициализация, тестовое окно.
2. Создание главного меню и docking-окон.
3. Панель иерархии сцены.
4. Панель инспектора с редактированием компонентов.
5. Обеспечение live preview (убедиться, что изменения видны).
6. Интеграция ImGuiizo и реализация манипуляторов.
7. Панель статистики.
8. Режимы Edit/Play.

ФОРМА И СОДЕРЖАНИЕ ОТЧЕТА

1. **Титульный лист** (ФИО, группа, название работы).
2. **Введение** (цель, задачи).
3. **Архитектура решения:**
 - Схема интеграции ImGui в игровой цикл.
 - Диаграмма взаимодействия между редактором и ECS.
 - Описание подсистемы гизмо.
4. **Реализация ключевых модулей:**
 - Код инициализации ImGui.
 - Пример создания инспектора для компонента Transform.

- Интеграция ImGuizmo (с фрагментом кода).
- Расчёт статистики.

5. Результаты:

- Скриншоты редактора с различными панелями.
- Скриншот с активным гизмо и выбранным объектом.
- Скриншот статистики.

6. Описание возникших трудностей и их решения.

7. Заключение (выводы, возможные улучшения).

ПРИЛОЖЕНИЯ К ОТЧЕТУ

- Ссылка на Git-репозиторий с полным кодом проекта.
- Видео-демонстрация работы редактора (опционально, но желательно).
- Конфигурационные файлы (если создавались).

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. В чём отличие immediate mode GUI от retained mode GUI? Почему ImGui популярен в инструментах разработки игр?
2. Как организовать передачу событий ввода между ImGui и игровым движком? Назначение флагов `io.WantCaptureMouse` и `io.WantCaptureKeyboard`.
3. Как устроен docking в ImGui? Какие флаги необходимо установить для его поддержки?
4. Каким образом изменения в инспекторе немедленно отражаются в окне предпросмотра (live preview)?
5. Как гизмо (ImGuizmo) преобразует действия мыши в изменения матрицы трансформации?
6. Какие данные необходимы для корректной работы 3D-манипулятора?
7. Как можно оптимизировать отрисовку панели иерархии при большом количестве объектов?
8. Какие метрики производительности полезно отображать в панели статистики? Как их рассчитать?
9. Как организовать переключение между режимами редактирования и игры? Что происходит с физикой и вводом в каждом режиме?

РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА

1. Официальная документация Dear ImGui – <https://github.com/ocornut/imgui> .
2. ImGuizmo – <https://github.com/CedricGuillemet/ImGuizmo>.

3. **Game Engine Architecture** – Jason Gregory (глава по инструментам разработки).
4. **Серия видео "Scion2D Game Engine"** – практическая интеграция ImGui в C++ движок.
5. **Статья "Immediate Mode GUI vs Retained Mode"** – различные ресурсы.
6. **Примеры интеграции ImGui** – в официальном репозитории, папка examples/ .

P.S.

Качественно реализованный редактор — основа для финального проекта и значительный плюс при защите. Уделите внимание эргономике интерфейса и стабильности live preview.